CSE113: Parallel Programming
Homework 2: Mutexes

Omar Fernandez
omfernan@ucsc.edu

October 18, 2026

Machine Ubunutu VM	Apple m5
Number of Cores	4

1 Part 1: Implementing Mutexes

1.1 Design

The design of these mutexes I took reference from the textbook in section 2.4 - 2.6 for the bakey mutex and the filter mutex.The program calls init on a mutex the user runs in the command line, then repeatedly calls lock and unlock for 1 second, and reports the throughput and per thread in the histogram.

Filter lock (`filter.h`). The Filter algorithm takes N- thread generalizations of Peterson's algorithm that works for $n > 2$ threads.It creates $n - 1$ "waiting rooms," called levels, that a thread must traverse before acquiring the lock. The levels are depicted in important properties:

- at least one thread trying to enter level l succeeds.
- if more than one thread is trying to enter level l, then at least one is blocked

The filter lock uses a boolean flag[] to indicate whether a thread can enter the critical section. The Filter lock generalizes this notion with an array of n-element integer level[], where the value of level[A] indicates the highest level that thread A is trying to enter. Level[] and Flag[] both use a atomic int array to maintain atomicity.The filter lock follows mutual exclusion , starvation free, and is deadlock free.

- **Constructor**: purpose is set number of threads to zero/
- `init(int num_threads)`: creates level and victim array for N threads, while storing a 0 for each level and victim thread created
- `lock(int tid)`: Each thread must pass through n 1 levels of "exclusion" to enter its critical section. Each level has a distinct victim[i] field used to "filter out" one thread, excluding it from that level unless no thread is at that level or higher. If there is a conflict the lock will continue spin as long as there exist a conflict.
- `unlock(int tid)`: level thread id is stored to zero allowing it to be used again.

Bakery lock (`bakery.h`). Lamports Bakery algorithm used a atomic boolean flag[] and a atomic int label[]. Flag[A] indicated whether A wants to enter a critical section and the label[A] is a integer that indicates the thread relative order when entering a bakery for each thread A. By a label[A] = max of the previous labels obtained to prevent which label was acquired first.

- **Constructor**: Sets number of threads to zero.
- `init(int num_threads)`: creates a flag[number of threads] and a label[number of threads], then stores(0) to each thread created.
- `lock(int tid)`: flag[thread id] sets it to true. Then creates a $label[A] = max(label[0], ..., label[n-1]) + 1$; to find the new current max. Then is the spin-wait from Lamport's Bakery algorithm: thread i busy-waits until no other thread k is competing for the critical section (flag[k]) with a lexicographically smaller (ticket, id) pair, meaning i only proceeds once it has the lowest ticket (ties broken by thread ID).
- `unlock(int tid)`: sets the thread id flag to false.
- **Deadlock-free:** Some waiting thread A has the unique least $(label[A], A)$ pair, and that thread never waits for another thread.
- **First-come-first-served:** If A's doorway precedes B's doorway, then A's label is smaller than B's. This holds because

$$write_A(label[A]) \rightarrow read_B(label[A]) \rightarrow write_B(label[B]) \rightarrow read_B(flag[A]),$$

  so B is locked out while $flag[A]$ is true.
- **Starvation-free:** This follows immediately from Lemmas 2.7.1 and 2.7.2, because any deadlock-free, first-come-first-served lock algorithm is also starvation-free.
- **mutual exclusion** therefore this algorithm is mutually exclusive

Yield variants. this_thread::yield() was used in both mutexes inside there locks.

Correctness testing. Data races in my mutexes because I lacked to init the threads to zero and I was passing the wrong argument of thread number to each mutex. As well I ran into an issue where in my virtual environment with ThreadSanitzer would not correctly work with the current gcc compiler. By lowering the address randomization this solved the FATAL Error.

```
FATAL: ThreadSanitizer: unexpected memory mapping 0x5c31578da000-0x5c31578dc000
Solution: $sudo sysctl vm.mmap_rnd_bits=28
```

```
. Filter lock test printed "total number: 0" inside Docker
- Not a lock bug: the Docker run used the TSan build
  (-fsanitize=thread), the non-Docker run did not.
- TSan slows the filter lock heavily (~n^2 atomic loads per
  acquisition: n-1 levels, each scanning n threads).
- The harness counts nothing until 500 warmup acquisitions,
  and the test runs only 1 second; with 8 threads under TSan
  the warmup never finished, so all counts stayed 0.
```

```
- Evidence: no TSan race reports, and nonzero counts with
  2 and 4 threads.
- Takeaway: use the TSan build for correctness at small thread
  counts, and the non-TSan build for performance numbers.
```

1.2 Results

Results were gathered *without* the thread sanitizer. Throughput is from a single run per configuration; for the 16-thread experiments, runs in which threads obtained the lock zero times or only single-digit times were discarded and rerun. CoV is the coefficient of variation of the per-thread lock-acquisition counts (CoV $= \sigma/\mu$, population standard deviation); lower means fairer.

Table 1: Throughput (total lock/unlock operations in 1 second) and fairness (CoV) for each mutex. $N = 4$ cores.

Mutex	**Threads**	**Throughput**	**CoV**	
C++ mutex	N (4)	13,980,513	0.0043	
Filter	N (4)	3,569,686	0.0024	
Bakery	N (4)	5,497,225	0.0006	
Filter + yield	N (4)	3,339,852	0.0021	
Bakery + yield	N (4)	3,597,797	0.0003	
C++ mutex	$2N$ (8)	6,426,957	0.0170	
Filter	$2N$ (8)	431,054	0.0180	
Bakery	$2N$ (8)	3,636	0.6766	Two groups: 764 vs. 147
Filter + yield	$2N$ (8)	1,264,834	0.0325	
Bakery + yield	$2N$ (8)	1,486,166	0.0003	
C++ mutex	$4N$ (16)	6,493,259	0.0163	
Filter	$4N$ (16)	1,002	3.2628	One thread got 854
Bakery	$4N$ (16)	234	0.0331	
Filter + yield	$4N$ (16)	520,613	0.0312	
Bakery + yield	$4N$ (16)	935,125	0.00001	

Histogram data. Per-thread lock acquisition counts for one representative run of each configuration .

Table 2: Per-thread acquisitions, $N = 4$ threads.

Mutex	T0	T1	T2	T3
C++ mutex	3505861	3507265	3474910	3492477
Filter	889172	893678	893219	893617
Bakery	1374996	1373432	1373710	1375087
Filter + yield	835087	834672	832953	837140
Bakery + yield	899452	899636	899667	899042

Table 3: Per-thread acquisitions, $2N = 8$ threads.

Mutex	T0	T1	T2	T3	T4	T5	T6	T7
C++ mutex	789018	790025	809321	805066	833901	792585	807297	799744
Filter	54002	52730	52513	53152	54615	55040	53756	55246
Bakery	764	764	764	756	147	147	147	147
Filter + yield	153021	167099	160536	159543	152622	160277	159390	152346
Bakery + yield	185718	185718	185722	185828	185829	185753	185783	185815

Table 4: Per-thread acquisitions, $4N = 16$ threads.

Mutex	T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	T13	T14	T15
C++ mutex	403321	407171	425624	405996	407202	408118	404437	400171	402774	400298	400663	405436	409101	402210	396248	414489
Filter	10	11	10	9	10	10	10	10	10	9	10	10	9	10	10	854
Bakery	14	14	14	14	15	14	15	15	15	15	14	15	15	15	15	15
Filter + yield	33882	33355	31263	32987	32962	31952	31258	33879	33335	31267	33356	33323	31248	31207	32953	32386
Bakery + yield	58445	58446	58444	58446	58446	58446	58445	58445	58446	58445	58446	58445	58445	58445	58445	58445

Outliers. The outliers occurred only in the runs without yield. With more threads than cores ($2N$ and $4N$), a thread holding the lock or waiting next in line can be descheduled while other threads spin. The filter lock at $4N$ had one thread obtain 854 of 1,002 acquisitions while the others obtained about 10 each; removing that thread lowers the CoV from 3.37 to about 0.05. The bakery lock at $2N$ split into two groups (764 vs. 147 acquisitions), and at $4N$ some runs produced no acquisitions at all, so those runs and the single-digit filter runs were excluded from the averages. Adding yield removed these outliers: all yield runs were nearly perfectly fair (CoV ≤ 0.033), because waiting threads give up the CPU and let the thread they are waiting for run.

1.3 Analysis

At $N = 4$ threads (equal to the core count), the C++ mutex is far faster (13.98M lock/unlocks) than the bakery (5.50M) and filter (3.57M) locks. The C++ mutex uses a single atomic read-modify-write on one cache line and puts waiting threads to sleep, so waiting generates no coherence traffic. Our locks may not use hardware RMW instructions, so they must be built from many loads and stores on shared arrays. The bakery lock needs roughly $O(N)$ shared-memory operations per acquisition and each thread writes only its own `flag` and `label` entries, while the filter lock needs roughly $O(N^2)$ operations per acquisition and every thread writes the shared `victim` entries, so those cache lines move between cores. This is why bakery beats filter at N threads.

Once threads outnumber cores ($2N$ and $4N$, oversubscription), both spin locks collapse without yield (bakery 3.6K and 234; filter 431K and 1.0K), while the C++ mutex stays near 6.4M because its waiters sleep. Spinning threads occupy the CPUs while the lock holder, or the next thread in line, may be descheduled. The bakery lock suffers most because it serves threads in strict ticket order, so every waiter depends on one particular thread being scheduled. Adding yield lets spinners give up the CPU, which improved throughput dramatically (filter 1.26M at $2N$ and 521K at $4N$; bakery 1.49M at $2N$ and 935K at $4N$) and made the runs stable and nearly perfectly fair. With yield, bakery is faster than filter at every thread count. At N threads yield costs some throughput (bakery 5.50M to 3.60M, filter 3.57M to 3.34M), because each yield is a system call and no other threads need the CPUs. Even with yield, neither lock reaches the C++ mutex (6.49M at $4N$).fyeild

2 Part 2: A Fair Reader-Writer Lock

2.1 Design

My design approach to maintain more fairness towards the writes while maintaining mutual exclusion,starvation-free and deadlock-free.My solution uses a writer priority strategy with a writer counter numberto help prevent writer starvation. When a writer calls lock(), it increments num writers while holding a internal mutex lock which would prevent new readers from entering while a writer is waiting. The writer waits till there are no active readers and no active write. Then while there are no active readers or no active writers a writer = true and decreamenting num writers. This ensures writers only have access to the critical section, while no writer is active or waiting readers can access it as much as they want. All shared varibles are protected by a mutex, to prevent data races. While no writers are waiting, readers can enter after a single check of the conditions, so the design does not add extra waiting time to reader throughput.

Function descriptions.

- `reader_lock()`: Allows readers to enter if no writer is active or waiting.
- `reader_unlock()`: Decreases the number of active readers when they finish.
- `writer_lock()`: Waits until no readers or writers are active, then grants exclusive access.
- `writer_unlock()`: Releases the writer's exclusive access.

2.2 Results

Results were collected both on my own machine and in the course Docker container. ISSUE: Mac has issues running local without docker container local results will not be used.

Table 5: Reader and writer throughput for the default and fair mutexes.

Environment	Config (R/W)	Mutex	Reader Throughput	Writer Throughput
Docker	6 / 2	Default	375252	47298
Docker	6 / 2	Fair	238429	417549
Docker	8 / 0	Default	4870767	0
Docker	8 / 0	Fair	4694482	0

Graph: default (6/2) configuration. Replace the placeholder values below with your measured numbers.

2.3 Analysis

- How much writer throughput increased with the fair mutex, and the cost to reader throughput? Cost of reader throughput was -3.6$, while the writer thoroughput increased to a whopping +782.8% (about 8.8x)
- Why the default mutex starves writers (continuous stream of overlapping readers). Because its continously allows readers to enter the crictal sections favoring readers rather than writers.

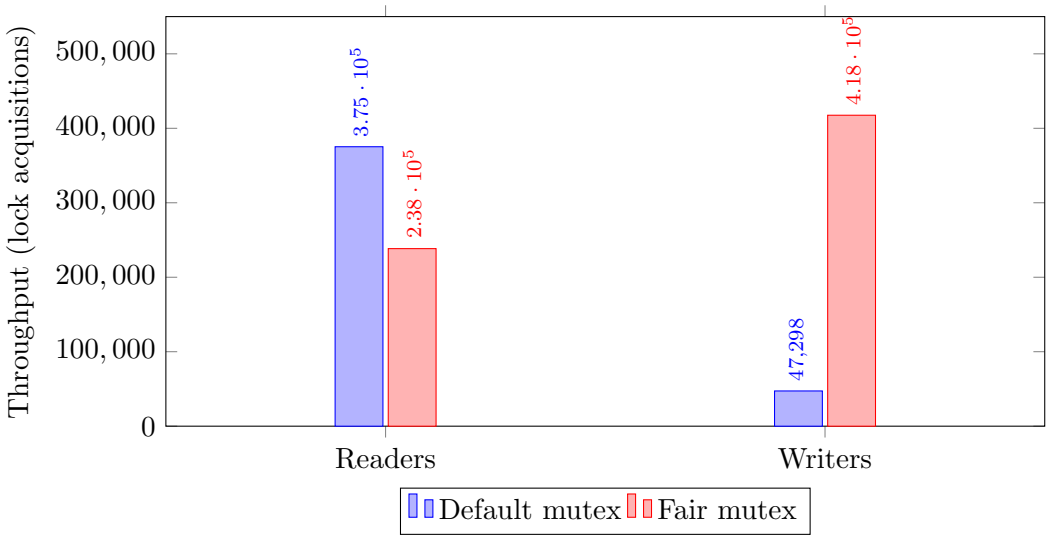


Figure 1: Reader and writer throughput with 6 readers and 2 writers. The fair mutex increases writer throughput by 782.8% at a cost of 36.5% of reader throughput.

If a writer is starving the reader doesn't care it will entering the critcal section as much as it wants to.

- That the 8/0 configuration shows your fair mutex does not significantly reduce reader throughput when there are no writers. Because when there are 0 writes there is nothing to write, meaning readers are just constanly reading back and forth in the mutex lock. The muxtex has fairness for writers, meaning as long as there isnt a writer starving any amount of readers can enter.
- Any unexpected results. Local machine does not like running the program I get errors, professor said not to include local machine results for mac.

3 Part 3: A Concurrent Linked List

3.1 Design

Coarse-grained locking (`coarse_lock_stack.h`). I added a single `std::mutex` as a private member of the stack. Every public method (`push`, `pop`, and `peek`) acquires it on entry and releases it on exit, using `lock_guard` so it is released on every return path. This makes each operation atomic with respect to the others and trivially correct, since only one thread can touch the underlying data at a time. The cost is that all operations are serialized, including `peek`, which does not modify the stack.

RW locking (`rw_lock_stack.h`). I replaced the mutex with a `std::shared_mutex`. `push` and `pop` modify the stack, so they take the exclusive lock (`lock`/`unlock`). `peek` only reads the top element, so it takes the shared lock (`lock_shared`/`unlock_shared`), which lets any number of peeking threads run at the same time. Since peeks are the most common operation in the benchmark, this is where concurrency should be gained.

SwapTop (`swaptop_stack.h`). SwapTop first acquires the read lock and inspects the top of the stack. If the stack is empty, or the top value already makes the swap unnecessary, the operation returns while still holding only the read lock, so the common case runs concurrently with peeks and other SwapTops. Only when a change is actually required does it release the read lock and acquire the write lock. A `shared_mutex` cannot upgrade atomically, so another thread may push, pop, or swap in the window between releasing and acquiring. To avoid acting on stale information, SwapTop re-reads the stack under the write lock and re-checks both the empty case and the swap condition before modifying anything. If the stack became empty, or another thread already performed the change, it does nothing. Because only the values 0, 1, and 2 are possible, the check is a cheap comparison, and the write lock is held only for that short re-check and update.

3.2 Results

Each experiment was run 3 times; the table reports the average throughput (total operations completed in the benchmark run). For SwapTop, the first row counts push, pop, and peek operations only, and the second row counts SwapTop operations.

Table 6: Throughput of each stack implementation (average of 3 runs).

Implementation	Metric	Run 1	Run 2	Run 3	Average
Coarse-grained	Push / Pop / Peek throughput	1,097,839	941,597	1,038,949	1,026,128
RW lock	Push / Pop / Peek throughput	1,043,815	734,286	924,181	900,761
SwapTop	Push / Pop / Peek throughput	1,946,804	1,404,049	1,925,135	1,758,663
SwapTop	SwapTop throughput	354,881	112,370	266,968	244,740

Spread of values. Run-to-run variation is noticeable in every configuration. For the coarse-grained stack the runs range from 941,597 to 1,097,839 (a range of 156,242, about 15% of the mean). The RW stack varies more, from 734,286 to 1,043,815 (a range of 309,529, about 34% of the mean). SwapTop total operations range from 1,516,419 to 2,301,685 (about 39% of the mean), and its SwapTop count varies the most, from 112,370 to 354,881 (about 99% of the mean). In every implementation, Run 2 is the lowest, so it acts as an outlier.

3.3 Analysis

Contrary to what one might expect, the RW lock did not outperform coarse-grained locking here. Its average throughput was 900,761 operations versus 1,026,128 for the coarse lock, about 12% lower, even though roughly 69% of operations in both benchmarks are peeks. The ranges of the two overlap (the best RW run, 1,043,815, beat two of the coarse runs), so the gap is partly within noise, but there is certainly no clear win for RW. Several factors explain this. A `peek` critical section is only a few instructions, so allowing readers to overlap saves very little time, while `shared_mutex` is heavier than `mutex` because each lock and unlock does extra bookkeeping on shared state. Every reader also performs an atomic update on the same reader count, so readers contend on that cache line even though they never conflict logically. Finally, the roughly 31% of operations that are pushes and pops need the exclusive lock, which must wait for all readers to drain and blocks new readers while held, so the concurrency gained by readers is partially lost.

SwapTop achieved the highest throughput, about 1.76 million push, pop, and peek operations plus about 245 thousand swaps per run. However, this is not a direct comparison with the other two

stacks, because the operation mix is very different: pushes and pops make up under 1% of operations and peeks about 84%, so almost nothing needs the write lock. That favors the RW design, and it shows that reading under the shared lock and only upgrading when a change is needed keeps the write lock rarely held. The weakness is that SwapTop itself varies enormously (112,370 to 354,881 swaps). Likely causes are thread scheduling and how many cores the machine gives the program, contention with other users on the shared server, and bursts where many threads need the write lock at once, which blocks the readers. The consistently low Run 2 across all implementations suggests an external factor, such as other jobs running on the machine, rather than the code itself. An unexpected result is that RW locking was slower than the coarse lock despite a read-heavy workload, which suggests that for very short critical sections, lock overhead matters more than reader concurrency.